

Unidad 5

Diseño de Interfaz de Usuario

Proceso de prototipado, herramientas colaborativas, IA aplicada y validación iterativa de prototipos móviles

Asignatura: Diseño de Software de Dispositivos Móviles

Carrera: Tecnicatura Universitaria en Diseño de Software — FTyCA, UNCA

Docente: Mg. Ing. Carlos Acosta Parra

Actividad asociada: Desarrollo de prototipo interactivo de baja o media fidelidad + video breve de recorrido

Objetivos de aprendizaje

Al finalizar esta unidad, el estudiante será capaz de:

1. Explicar el proceso de prototipado de una aplicación móvil desde bocetos y wireframes hasta prototipos de alta fidelidad.
2. Distinguir niveles de fidelidad, alcance y propósito de un prototipo según el momento del proyecto.
3. Utilizar criterios de diseño visual para convertir una idea validada en pantallas móviles consistentes, comprensibles y accesibles.
4. Seleccionar herramientas digitales y prácticas colaborativas adecuadas para diseñar, documentar y compartir prototipos.
5. Incorporar inteligencia artificial como apoyo al diseño sin delegar en ella las decisiones críticas sobre usuarios, accesibilidad y valor de producto.
6. Planificar pruebas con usuarios, interpretar hallazgos y priorizar mejoras iterativas antes del desarrollo funcional.

Introducción

En la unidad anterior se trabajó sobre la pregunta estratégica: qué producto móvil vale la pena construir, para quién y por qué. La unidad 5 avanza hacia una pregunta más concreta: cómo hacemos visible, navegable y evaluable esa idea antes de invertir esfuerzo de desarrollo.

Diseñar una interfaz de usuario no consiste solamente en elegir colores o ubicar botones. En aplicaciones móviles, la interfaz concentra la promesa del producto: muestra qué acciones son posibles, qué información es importante, qué camino debe seguir el usuario y qué tan confiable se percibe el sistema. Un buen prototipo permite discutir todo eso cuando todavía es barato cambiar de dirección.

El prototipado es una técnica de aprendizaje. Un prototipo no es una versión incompleta de la aplicación final, sino una representación deliberadamente parcial que sirve para responder preguntas: ¿el usuario entiende el flujo?, ¿encuentra la acción principal?, ¿la navegación coincide con su modelo mental?, ¿la propuesta de valor se percibe en los primeros segundos?, ¿la interfaz se puede usar con una mano?, ¿funciona para personas con distintas capacidades visuales o motoras?

Idea central

Un prototipo no se mide por lo parecido que es a una app terminada, sino por la calidad de las decisiones que permite tomar.

1. El prototipo como instrumento de diseño

Un prototipo es una simulación de una solución futura. Puede ser tan simple como una secuencia de pantallas dibujadas en papel o tan sofisticado como una interfaz navegable con transiciones, microinteracciones y datos de ejemplo. En todos los casos, su valor depende de una relación clara entre pregunta, costo y evidencia.

En productos móviles, prototipar es especialmente importante porque el espacio disponible es reducido, las sesiones de uso suelen ser breves, las interrupciones son frecuentes y muchas interacciones dependen del tacto, los gestos y el contexto físico del usuario. Un flujo que parece claro en una

computadora puede volverse confuso cuando debe resolverse en una pantalla pequeña, caminando, con mala conectividad o usando una sola mano.

1.1 Propósitos principales

- Explorar alternativas de solución sin comprometerse todavía con una arquitectura técnica.
- Comunicar una idea de producto a docentes, compañeros, clientes o usuarios potenciales.
- Alinear al equipo sobre flujos, pantallas, jerarquía de información y criterios visuales.
- Detectar problemas de comprensión antes de escribir código.
- Reducir incertidumbre sobre una hipótesis de valor, usabilidad o viabilidad.
- Preparar el traspaso al desarrollo con especificaciones visuales más claras.

1.2 Preguntas que un prototipo puede responder

Tipo de pregunta	Ejemplo	Prototipo más adecuado
Valor	¿La solución propuesta resuelve un problema real y relevante?	Boceto, storyboard, landing de prueba o flujo de baja fidelidad.
Comprensión	¿El usuario entiende qué debe hacer en cada pantalla?	Wireframe navegable de baja o media fidelidad.
Navegación	¿El recorrido entre pantallas coincide con la tarea del usuario?	Prototipo interactivo con hotspots y estados básicos.
Diseño visual	¿La jerarquía, el contraste y los componentes comunican correctamente?	Mockup o prototipo de alta fidelidad.
Interacción	¿Las transiciones, gestos y respuestas del sistema son claras?	Prototipo de alta fidelidad con microinteracciones.
Viabilidad	¿La solución requiere datos, permisos o integraciones difíciles?	Prototipo técnico o spike funcional acotado.

1.3 Fidelidad: baja, media y alta

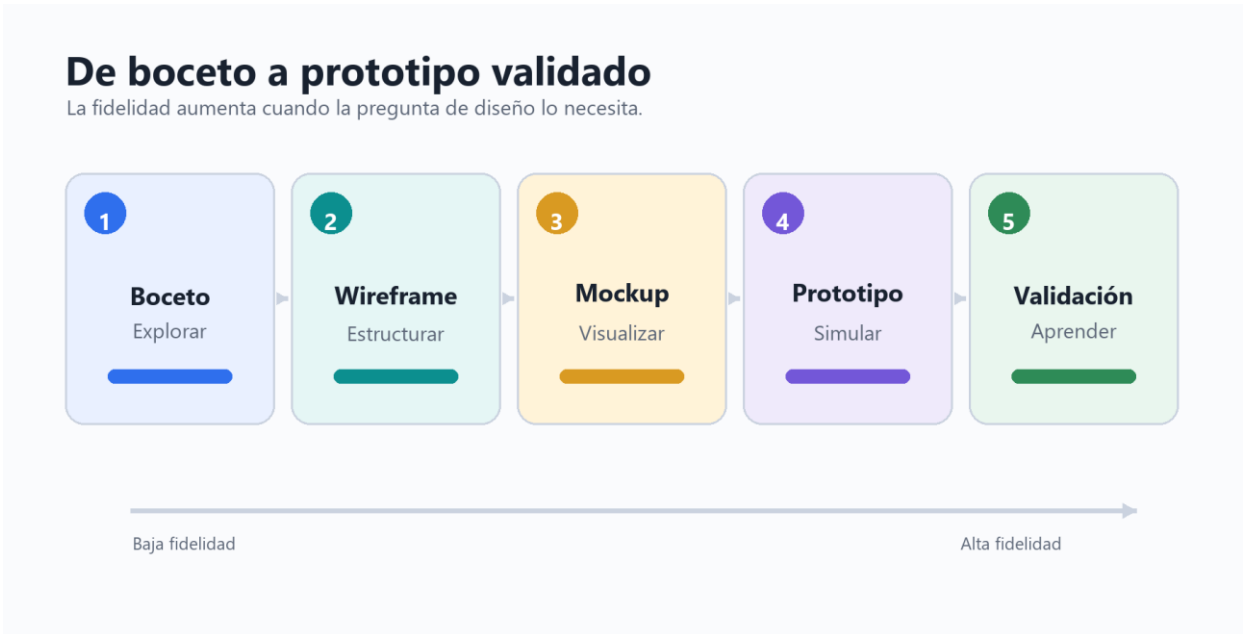


Figura 1. Progresión de fidelidad desde un boceto exploratorio hasta un prototipo validado.

La fidelidad describe cuánto se parece el prototipo al producto final en términos visuales, funcionales y de contenido. No existe un nivel universalmente superior: cada nivel responde a necesidades distintas.

Nivel	Características	Ventajas	Riesgos
Baja fidelidad	Bocetos en papel, wireframes simples, cajas y textos aproximados.	Rápida, barata, invita a discutir estructura sin apego estético.	Puede ocultar problemas de contenido real, contraste o densidad visual.
Media fidelidad	Pantallas digitales con layout, jerarquía y navegación, pero sin pulido final.	Ideal para validar flujos y tareas; suficiente para pruebas tempranas.	Puede parecer más terminado de lo que realmente está.
Alta fidelidad	Interfaz visual cercana a producción, componentes, colores, tipografía, estados y transiciones.	Permite evaluar percepción, accesibilidad y handoff a desarrollo.	Costosa; puede generar apego a decisiones todavía no validadas.

Regla práctica

Usar la menor fidelidad capaz de responder la pregunta actual. Si todavía se debate el flujo, no conviene invertir horas en sombras, íconos o animaciones.

1.4 Alcance del prototipo

El alcance define qué parte del producto se representa. En una app móvil real no suele ser necesario prototipar todas las pantallas al mismo nivel. Conviene identificar el camino crítico: la secuencia mínima que permite al usuario obtener valor.

- Vertical slice: representa un flujo completo de extremo a extremo, aunque con pocas funciones. Ejemplo: registrarse, buscar un turno, confirmarlo y ver el comprobante.
- Horizontal slice: muestra muchas secciones de la app, pero con poca profundidad. Sirve para discutir arquitectura de información.
- Prototipo de componente: se enfoca en un elemento específico, como un calendario, selector de ubicación, formulario de pago o pantalla de permisos.
- Prototipo de estado: representa variaciones como carga, error, vacío, éxito, sin conexión y permiso denegado.

2. Del wireframe al prototipo de alta fidelidad

El proceso de prototipado puede organizarse como una progresión: bocetar, estructurar, conectar, visualizar, simular y validar. No siempre es lineal; los equipos vuelven hacia atrás cuando una prueba muestra que el flujo no funciona o cuando aparece una restricción técnica.

2.1 Bocetos iniciales

El boceto es la forma más rápida de pensar con las manos. Permite producir muchas alternativas antes de decidir. En el contexto de una materia de diseño de software, el boceto no busca talento artístico: busca claridad de intención.

- Definir primero la tarea principal de la pantalla.
- Representar solo bloques esenciales: encabezado, contenido, acción principal, navegación, estados clave.
- Dibujar varias alternativas pequeñas antes de ampliar una.
- Anotar dudas y supuestos junto al boceto.
- Evitar detalles decorativos que distraigan de la estructura.

2.2 Wireframes



Figura 2. Zonas funcionales principales que conviene resolver en un wireframe móvil.

Un wireframe es una representación estructural de una pantalla. Muestra qué elementos existen, dónde se ubican y qué prioridad tienen, pero todavía no define la identidad visual completa. En móvil, el wireframe obliga a decidir qué entra, qué se posterga y qué se elimina.

Elemento	Pregunta de diseño	Decisión esperada
Encabezado	¿Dónde se ubica el usuario y cómo vuelve atrás?	Título claro, navegación coherente con la plataforma.
Contenido principal	¿Cuál es la información que permite avanzar?	Jerarquía visual y agrupación por proximidad.
Acción primaria	¿Qué espera la app que el usuario haga ahora?	Botón o gesto principal visible y diferenciado.
Acciones secundarias	¿Qué opciones existen sin competir con la principal?	Menús, enlaces o botones de menor peso visual.
Feedback	¿Cómo responde el sistema a una acción?	Estados de carga, confirmación, error o vacío.
Accesibilidad	¿Puede usarse con distintos tamaños de texto y áreas táctiles?	Controles legibles, contrastes adecuados y objetivos táctiles suficientes.

2.3 Flujos de usuario

Un flujo de usuario describe la secuencia de pasos que una persona sigue para cumplir una tarea. Antes de diseñar pantallas aisladas, conviene representar el recorrido completo. Esto evita interfaces bonitas pero desconectadas del objetivo.

- Punto de entrada: desde dónde llega el usuario, por ejemplo notificación, buscador, pantalla de inicio o enlace compartido.
- Decisión principal: qué elección debe tomar para avanzar.
- Pantallas intermedias: formularios, selección, revisión, permisos o confirmaciones.
- Resultado: éxito, error, espera, cancelación o alternativa.
- Recuperación: qué ocurre si el usuario se equivoca, no tiene conexión o abandona el proceso.

Ejemplo móvil

En una app de turnos médicos, el flujo no es simplemente 'ver calendario'. El camino real puede incluir elegir especialidad, seleccionar profesional, filtrar cobertura, pedir permiso para notificaciones, confirmar el turno y recibir instrucciones previas.

2.4 Mockups y alta fidelidad

El mockup de alta fidelidad traduce el wireframe a una interfaz visual cercana al producto final. Aquí aparecen decisiones de tipografía, color, iconografía, espaciado, componentes, densidad de información, estados y comportamiento visual.

- Usar contenido representativo, no texto falso cuando el contenido real afecte el diseño.
- Definir una jerarquía tipográfica con pocos niveles y aplicarla de manera consistente.
- Crear componentes reutilizables: botones, campos, tarjetas, barras de navegación, modales, chips, listas.
- Diseñar estados de interacción: normal, presionado, deshabilitado, foco, carga, error y éxito.
- Verificar contraste, tamaño de toque y legibilidad en dispositivos pequeños.
- Preparar pantallas para casos límite: lista vacía, error de red, permiso denegado, texto largo y usuario sin datos.

2.5 Prototipos interactivos

Un prototipo interactivo conecta pantallas mediante zonas clicables, botones, gestos y transiciones. No ejecuta lógica real, pero simula la experiencia suficiente para que un usuario realice tareas durante una prueba.

Recurso	Uso correcto	Cuidado
Hotspots	Conectar botones, tarjetas o áreas táctiles con otra pantalla.	No convertir toda la pantalla en un enlace; oculta errores de interacción.
Overlays	Mostrar menús, diálogos, filtros o confirmaciones sin cambiar de pantalla.	Evitar superposiciones que no reflejen el patrón de la plataforma.
Transiciones	Comunicar continuidad espacial o cambio de estado.	No abusar de animaciones largas que retrasan la tarea.
Variables/estados	Simular selección, pestañas, checkboxes o contenido condicional.	Mantener el prototipo comprensible para quien lo revise.

Recurso	Uso correcto	Cuidado
Datos de ejemplo	Hacer creíble el escenario de uso.	No usar datos sensibles ni identidades reales sin consentimiento.

3. Criterios de diseño visual para interfaces móviles

La interfaz móvil combina restricciones técnicas, percepción visual y comportamiento humano. Un prototipo de alta fidelidad debe demostrar que la propuesta puede ser usada con claridad, no solo que se ve atractiva.

3.1 Jerarquía visual

La jerarquía visual ordena la atención. En una pantalla móvil, el usuario necesita reconocer en segundos qué es importante, qué puede tocar y qué ocurrirá después.

- Tamaño: los títulos, valores críticos y acciones principales deben destacarse sin saturar.
- Peso y contraste: usar negrita y color con moderación para reforzar prioridad.
- Espacio: agrupar elementos relacionados y separar secciones distintas.
- Posición: ubicar la acción principal donde sea fácil alcanzarla y descubrirla.
- Repetición: mantener patrones consistentes para que el usuario no reaprenda cada pantalla.

3.2 Sistemas de diseño y tokens

Un sistema de diseño documenta decisiones reutilizables. En proyectos pequeños puede ser una página con colores, tipografías, botones y componentes; en organizaciones grandes puede incluir bibliotecas completas, documentación, reglas de accesibilidad y código.

Concepto	Descripción	Ejemplo
Token de diseño	Variable que guarda una decisión visual reutilizable.	color.primary, spacing.16, radius.medium.
Componente	Elemento reutilizable con estructura y comportamiento.	Botón primario, campo de texto, tarjeta de producto.
Variante	Configuración de un componente según contexto.	Botón pequeño/grande, habilitado/deshabilitado, con ícono/sin ícono.
Patrón	Solución recurrente para una necesidad de interacción.	Búsqueda con filtros, navegación inferior, flujo de onboarding.
Guía de uso	Regla que explica cuándo usar o evitar un componente.	No usar un diálogo para acciones de baja importancia.

3.3 Accesibilidad desde el prototipo

La accesibilidad no debe quedar para el final del desarrollo. Un prototipo puede y debe anticipar decisiones que harán la aplicación usable por más personas.

- Contraste suficiente entre texto y fondo, especialmente en botones y estados de error.
- Tamaño táctil cómodo para dedos, no para cursor de mouse.
- Textos claros para lectores de pantalla: los íconos sin etiqueta visual deben tener nombre accesible previsto.

- No depender solo del color para comunicar estados; combinar color con texto, ícono o patrón.
- Soportar texto dinámico: prever qué ocurre si el usuario aumenta el tamaño de fuente.
- Evitar gestos únicos difíciles de descubrir; ofrecer alternativas visibles cuando la acción sea importante.

Criterio mínimo

Si un prototipo solo funciona cuando el texto es pequeño, el color se percibe perfectamente y el usuario conoce gestos ocultos, todavía no está listo para pasar a desarrollo.

3.4 Estados que no se deben olvidar

Estado	Qué debe mostrar	Por qué importa
Carga	Indicador de progreso, skeleton o mensaje breve.	Evita toques repetidos y comunica que el sistema trabaja.
Vacío	Explicación y próxima acción.	Una lista vacía sin orientación parece un error.
Error	Causa comprensible y forma de recuperación.	Reduce frustración y soporte innecesario.
Sin conexión	Mensaje, acciones disponibles y reintento.	El móvil depende de redes variables.
Permiso denegado	Motivo del permiso y alternativa.	Cámara, ubicación o notificaciones requieren confianza.
Éxito	Confirmación clara y siguiente paso.	Cierra la tarea y orienta la continuidad.

4. Herramientas digitales y trabajo colaborativo

Las herramientas de diseño modernas no son solo lienzo digitales. Funcionan como entornos colaborativos donde se investiga, boceta, diseña, comenta, prototipa, versiona y entrega información al equipo de desarrollo.

4.1 Categorías de herramientas

Categoría	Uso principal	Ejemplos
Pizarras colaborativas	Mapas de empatía, flujos, lluvia de ideas, priorización.	FigJam, Miro, Mural.
Diseño de interfaz	Wireframes, mockups, componentes y prototipos.	Figma, Penpot, Sketch.
Prototipado avanzado	Microinteracciones, animaciones complejas, prototipos cercanos a producción.	ProtoPie, Framer.
Handoff a desarrollo	Medidas, estilos, assets, especificaciones, comentarios.	Dev Mode de Figma, Zeplin, documentación del design system.
Pruebas con usuarios	Grabación, tareas, feedback remoto, métricas.	Maze, Useberry, Lookback, sesiones moderadas por videollamada.

Categoría	Uso principal	Ejemplos
Gestión del trabajo	Historias de usuario, tareas, seguimiento de decisiones.	Trello, Jira, GitHub Projects, Notion.

4.2 Buenas prácticas de colaboración

- Nombrar páginas y frames con una convención clara: 01_Exploración, 02_Wireframes, 03_Prototipo, 04_Entrega.
- Separar exploraciones descartadas de la versión vigente para evitar confusiones.
- Usar comentarios vinculados a puntos concretos de la interfaz.
- Mantener una biblioteca mínima de componentes compartidos.
- Registrar decisiones importantes: por qué se eligió un flujo, qué hipótesis valida y qué evidencia lo respalda.
- Diseñar junto con desarrollo cuando haya dudas sobre permisos, datos, rendimiento o componentes nativos.

4.3 Handoff: del diseño al desarrollo

El handoff es el traspaso de diseño a desarrollo. Un mal handoff produce interpretaciones inconsistentes, retrabajo y discusiones tardías. Un buen handoff no pretende reemplazar la conversación: la ordena.

- Pantallas finales del flujo principal y estados alternativos.
- Componentes reutilizables con variantes y reglas de uso.
- Medidas, espaciados, colores, tipografías e íconos exportables.
- Textos reales o criterios de redacción para mensajes, errores y botones.
- Especificación de navegación, transiciones y comportamiento de formularios.
- Notas sobre accesibilidad: etiquetas, orden de lectura, contraste y tamaño táctil.

Trabajo en equipo

El prototipo no debe circular como una imagen muda. Debe ir acompañado de contexto: problema, usuario, flujo, supuestos, criterios de validación y decisiones pendientes.

5. Inteligencia artificial aplicada al prototipado

La inteligencia artificial generativa puede acelerar partes del proceso de diseño, especialmente en exploración, variaciones visuales, redacción, análisis de feedback y generación de assets. Sin embargo, no reemplaza la investigación con usuarios ni el juicio de diseño. La IA propone; el equipo decide, valida y se hace responsable.

5.1 Usos útiles



Figura 3. Uso responsable de IA generativa como apoyo al proceso de diseño.

Momento	Cómo puede ayudar la IA	Resultado esperado
Ideación visual	Generar variaciones de layout, moodboards o estilos.	Opciones iniciales para discutir, no decisiones finales.
Contenido UX	Proponer textos de botones, mensajes de error, onboarding o microcopy.	Copys más claros que luego se ajustan al tono del producto.
Wireframes	Sugerir estructuras para una tarea o pantalla concreta.	Punto de partida para bocetar más rápido.
Assets	Crear imágenes de apoyo, avatares ficticios, íconos exploratorios.	Material no sensible para prototipos y pruebas.
Accesibilidad	Detectar posibles problemas de contraste, jerarquía o lenguaje.	Lista de revisión que debe confirmarse con herramientas y criterio humano.
Síntesis de pruebas	Agrupar comentarios de usuarios y encontrar patrones.	Hallazgos preliminares revisados por el equipo.

5.2 Prompting para diseño de interfaces

Para obtener resultados útiles, el pedido a la IA debe incluir contexto. Un prompt vago como 'diseña una app linda' produce respuestas genéricas. Un prompt de diseño debe indicar usuario, tarea, restricciones, plataforma, tono, nivel de fidelidad y formato de salida.

Elemento del prompt	Qué especificar	Ejemplo
Usuario	Perfil, contexto y necesidad.	Estudiantes universitarios que consultan horarios desde el celular.

Elemento del prompt	Qué especificar	Ejemplo
Tarea	Objetivo que debe poder completar.	Reservar un turno en menos de tres pasos.
Plataforma	iOS, Android, web móvil o multiplataforma.	Android con navegación inferior y componentes Material.
Restricciones	Pantalla, conectividad, accesibilidad, datos disponibles.	Debe funcionar con una mano y contemplar texto aumentado.
Estilo	Tono visual y verbal.	Institucional, claro, sobrio, con alto contraste.
Salida	Formato deseado.	Lista de pantallas, jerarquía de contenido y estados alternativos.

5.3 Riesgos y límites

- Homogeneización: muchas herramientas generan interfaces visualmente correctas pero parecidas entre sí.
- Falsa seguridad: una propuesta convincente puede no estar validada con usuarios reales.
- Sesgos: los ejemplos entrenados pueden no representar a poblaciones locales, adultos mayores, personas con discapacidad o contextos de baja conectividad.
- Privacidad: no se deben subir datos personales, historias clínicas, credenciales ni información sensible a herramientas externas sin autorización.
- Accesibilidad incompleta: la IA puede mencionar buenas prácticas y aun así producir una pantalla con contraste insuficiente o jerarquía confusa.
- Propiedad intelectual: conviene evitar copiar estilos de marcas o productos existentes y revisar licencias de assets generados.

Uso responsable

La IA es útil cuando amplía el espacio de alternativas y reduce tareas mecánicas. Se vuelve peligrosa cuando sustituye la comprensión del usuario o se acepta como autoridad sin pruebas.

6. Validación de prototipos con usuarios

Validar un prototipo significa observar si personas representativas pueden usarlo para completar tareas relevantes. No se trata de preguntar si 'les gusta', sino de obtener evidencia sobre comprensión, navegación, errores, fricciones y expectativas.

6.1 Qué validar

Aspecto	Pregunta	Evidencia
Comprensión	¿El usuario entiende de qué trata la pantalla?	Explicación espontánea, dudas, interpretaciones equivocadas.
Encontrabilidad	¿Ubica la acción o información que necesita?	Tiempo hasta el primer toque correcto, exploración, retrocesos.
Flujo	¿Completa la tarea sin intervención?	Tasa de éxito, abandonos, pasos innecesarios.
Confianza	¿La interfaz comunica seguridad y control?	Comentarios sobre permisos, pagos, datos personales o confirmaciones.
Accesibilidad	¿Puede interactuar sin esfuerzo excesivo?	Problemas de lectura, toque, foco, contraste o lenguaje.
Valor	¿La solución resulta útil en su contexto real?	Relato del usuario, comparación con alternativas, intención de uso.

6.2 Métodos de prueba

- Prueba moderada presencial: el facilitador observa y pregunta mientras el usuario realiza tareas.
- Prueba moderada remota: útil cuando los usuarios están distribuidos; permite grabar pantalla y audio.
- Prueba no moderada: la persona completa tareas sola mediante una plataforma; escala mejor pero da menos contexto.
- Prueba de guerrilla: sesiones breves con usuarios disponibles; sirve para detectar problemas evidentes en etapas tempranas.
- Test de los cinco segundos: muestra una pantalla por pocos segundos y pregunta qué recuerda el usuario.
- Card sorting o tree testing: evalúa categorías, navegación y arquitectura de información.

6.3 Guion básico de una prueba

1. Presentar el objetivo: se evalúa el prototipo, no a la persona.
2. Pedir consentimiento para tomar notas o grabar si corresponde.
3. Hacer preguntas de contexto: experiencia previa, frecuencia de uso, situación típica.
4. Entregar una tarea concreta: por ejemplo, 'reservá un turno para la semana próxima'.
5. Pedir que piense en voz alta mientras interactúa.

6. No guiar ni explicar la interfaz durante la tarea; registrar dónde se detiene.
7. Cerrar con preguntas abiertas: qué resultó claro, qué generó dudas, qué cambiaría.
8. Agradecer y sintetizar hallazgos sin discutir ni defender el diseño.

6.4 Métricas útiles

Métrica	Qué mide	Cómo interpretarla
Éxito de tarea	Porcentaje de usuarios que completan la tarea.	Es la señal principal de usabilidad del flujo.
Tiempo de tarea	Duración hasta completar el objetivo.	Mucho tiempo puede indicar fricción o baja encontrabilidad.
Errores	Toques equivocados, retrocesos, campos mal completados.	Revela problemas de affordance, copy o arquitectura.
SEQ	Single Ease Question: percepción de facilidad tras una tarea.	Complementa observación con percepción subjetiva.
SUS	System Usability Scale para evaluación global.	Útil cuando hay prototipos más maduros o comparaciones.
Comentarios cualitativos	Dudas, expectativas, lenguaje del usuario.	Explican el porqué detrás de las métricas.

6.5 Priorización de hallazgos

Después de una prueba, no todos los hallazgos tienen la misma importancia. Conviene priorizar por severidad, frecuencia y relación con el objetivo principal.

Severidad	Descripción	Acción
Crítica	Impide completar la tarea o genera riesgo importante.	Corregir antes de seguir validando o desarrollar.
Alta	Produce errores frecuentes o mucha confusión.	Corregir en la siguiente iteración.
Media	Genera fricción pero el usuario logra avanzar.	Ajustar si afecta al flujo principal.
Baja	Preferencia estética o mejora menor.	Registrar y evaluar más adelante.

7. Mejora iterativa

La iteración convierte el prototipo en aprendizaje acumulado. Cada ciclo debe tener una hipótesis, una versión del diseño, una prueba y una decisión. Iterar no significa cambiar cosas al azar: significa mejorar con evidencia.

7.1 Ciclo recomendado



Figura 4. Ciclo de validación para convertir hipótesis de diseño en mejoras concretas.

1. Definir hipótesis: qué creemos que ocurrirá y por qué.
2. Construir la versión mínima del prototipo que permita probarla.
3. Probar con usuarios o revisores representativos.
4. Registrar observaciones sin reinterpretarlas prematuramente.
5. Agrupar patrones y separar hechos de opiniones.
6. Priorizar cambios según impacto y esfuerzo.
7. Actualizar el prototipo y documentar la decisión.

7.2 Errores frecuentes

- Pedir opiniones generales en lugar de observar tareas concretas.
- Probar con compañeros que ya conocen la consigna y no representan al usuario.
- Defender el diseño durante la prueba, explicando lo que la interfaz no comunica sola.
- Cambiar todo después de una sola opinión aislada.
- Validar solo la pantalla principal y olvidar estados de error, vacío o permisos.
- Pasar a alta fidelidad demasiado pronto y generar apego a una solución débil.
- Confundir estética con usabilidad: una interfaz puede gustar y aun así fallar.

7.3 Documentación de decisiones

Una buena práctica es mantener un registro breve de decisiones de diseño. Esto evita discusiones repetidas y ayuda a justificar el prototipo en una entrega académica.

Campo	Contenido sugerido
Decisión	Qué se cambió o se eligió.
Motivo	Qué problema u objetivo justifica la decisión.
Evidencia	Observación de usuarios, criterio de accesibilidad, restricción técnica o pauta de plataforma.
Alternativas	Qué opciones se descartaron y por qué.
Pendiente	Qué falta validar o implementar.

8. Guía para el trabajo práctico de la unidad

La actividad de la unidad consiste en desarrollar un prototipo interactivo de baja o media fidelidad y entregar un video breve de recorrido. El objetivo es demostrar que la idea de producto puede convertirse en un flujo usable, comprensible y mejorable.

8.1 Entregables sugeridos

- Archivo o enlace al prototipo interactivo.
- Breve descripción del usuario objetivo y problema abordado.
- Mapa del flujo principal prototipado.
- Pantallas del flujo principal y estados alternativos relevantes.
- Video de recorrido: máximo recomendado de 3 a 5 minutos, mostrando el objetivo, el flujo y las decisiones principales.
- Resumen de una validación simple: usuarios consultados, tarea probada, hallazgos y mejoras aplicadas.

8.2 Alcance mínimo recomendado

Elemento	Mínimo esperado	Mejor práctica
Pantallas	Entre 5 y 8 pantallas conectadas.	Incluir inicio, tarea principal, confirmación y un estado alternativo.
Interacción	Botones o zonas clicables que permitan recorrer el flujo.	Evitar enlaces invisibles o rutas que no existen.
Contenido	Textos suficientemente reales para entender la experiencia.	Usar datos ficticios pero plausibles.
Consistencia	Mismos estilos para acciones similares.	Crear componentes reutilizables.
Accesibilidad	Contraste, legibilidad y tamaño táctil razonables.	Revisar texto dinámico y no depender solo de color.
Validación	Al menos una prueba informal con usuario o par externo al equipo.	Registrar observaciones y mostrar una mejora concreta.

8.3 Checklist antes de entregar

OK	Criterio
	El flujo principal puede recorrerse de inicio a fin sin explicación oral.
	La acción principal de cada pantalla se distingue de acciones secundarias.
	Los nombres de pantallas, botones y mensajes usan lenguaje del usuario.
	El prototipo incluye al menos un estado alternativo: error, vacío, carga, sin conexión o permiso.
	Los tamaños de texto y controles son legibles en una pantalla móvil.
	No se usaron datos personales reales en capturas, prototipo o video.
	El video muestra el problema, el usuario, el recorrido y una decisión de diseño justificada.
	El equipo puede explicar qué se cambió luego de validar o revisar el prototipo.

9. Síntesis de la unidad

La unidad 5 conecta la ideación con el desarrollo. Una idea de aplicación móvil se vuelve discutible cuando se transforma en pantallas, flujos y estados. El prototipo permite ensayar esa experiencia antes de programarla, revelar contradicciones, comprobar hipótesis y mejorar la solución con evidencia.

El proceso comienza con representaciones de baja fidelidad, avanza hacia wireframes y prototipos interactivos, y puede llegar a mockups de alta fidelidad cuando ya existe suficiente claridad sobre el flujo. Las herramientas digitales facilitan la colaboración, pero la calidad del resultado depende de decisiones de diseño: jerarquía, consistencia, accesibilidad, contenido, feedback y adecuación a la plataforma.

La inteligencia artificial suma velocidad y variedad, aunque no reemplaza la validación con usuarios. Un prototipo debe ser evaluado mediante tareas reales, hallazgos priorizados y ciclos de mejora. Diseñar interfaz de usuario, en este contexto, es aprender antes de construir.

Bibliografía y recursos de consulta

- Apple. Human Interface Guidelines. <https://developer.apple.com/design/human-interface-guidelines/>
- Google. Material Design 3. <https://m3.material.io/>
- Figma Help Center. Prototyping in Figma. <https://help.figma.com/>
- World Wide Web Consortium (W3C). Web Content Accessibility Guidelines (WCAG) 2.2. <https://www.w3.org/TR/WCAG22/>
- Nielsen Norman Group. Usability testing, prototyping and mobile UX articles. <https://www.nngroup.com/articles/>

- Knapp, J., Zeratsky, J. y Kowitz, B. Sprint: How to Solve Big Problems and Test New Ideas in Just Five Days. Simon & Schuster, 2016.
- Norman, D. The Design of Everyday Things. Basic Books, edición revisada, 2013.